



Lab Report – EE-170L Computer Programming (Lab 02)

NAME	ABDUL AHAD
REG.NO	0692
SEMESTER	2nd
LAB	7
DATE	28/4/2026

1. Objectives

- Understand what a two-dimensional array is.
- Learn when to use two-dimensional arrays.
- Practice declaring and initializing 2D arrays.
- Access elements using row and column indices.

- Perform operations such as summing rows/columns and finding maximum values.

2. Theory

A **two-dimensional array** is the simplest form of a multidimensional array. It can be thought of as a table with rows and columns, where each element is identified by two indices: one for the row and one for the column. **Declaration Syntax:**

`type arrayName[rows][columns];` **For**

example:

```
int matrix[3][3];
```

This declares a 3x3 integer array.

Initialization:

Arrays can be initialized row by row using nested braces, or in a single list. For example:

```
int a[3][4] = { {0,1,2,3}, {4,5,6,7}, {8,9,10,11} };
```

Accessing

Elements:

Each element is accessed using two indices: `int`

```
val = a[2][3]; // element at row 2, column 3
```

Two-dimensional arrays are widely used in applications such as matrices, tables, and grids. They form the basis for advanced concepts like image processing, dynamic programming, and scientific simulations.

3. Lab Tasks and Programs

Task 1: Two-Dimensional Array Declaration and Initialization

```
#include <iostream>    // include input-output library using
namespace std;        // use standard namespace int main() { //
main function starts

int matrix[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} }; // declare and
initialize 3x3 array for(int i = 0; i < 3; i++) { // loop for
rows for(int j = 0; j < 3; j++) { // loop for columns

    cout << matrix[i][j] << " "; // print each element
}

cout << endl;           // move to next line after each row
}

return 0;               // end program
}
```

Task 2: Sum of Rows and Columns in a Two-Dimensional Array

```
#include <iostream> // include input-output library using
namespace std; // use standard namespace int main() { //
main function starts

int matrix[3][3] = { {2,4,6}, {1,3,5}, {7,8,9} }; // declare and
initialize 3x3 array

    int totalSum = 0; // variable for total sum for(int i = 0; i < 3; i++)
    { // loop for rows int rowSum = 0; // sum of current row
    for(int j = 0; j < 3; j++) { // loop for columns rowSum +=
    matrix[i][j]; // add element to row sum totalSum +=
    matrix[i][j]; // add element to total sum

    }

    cout << "Sum of row " << i << " = " << rowSum << endl; //display
row sum

    }

    for(int j = 0; j < 3; j++) { // loop for columns int colSum = 0;
    // sum of current column for(int i = 0; i < 3; i++) { // loop
    for rows colSum += matrix[i][j]; // add element to column
sum

    }

    cout << "Sum of column " << j << " = " << colSum << endl; //
```

```
display column sum
```

```
}
```

```
cout << "Total sum = " << totalSum << endl; // display total sum
```

```
return 0; // end program
```

```
}
```

Task 3: Finding the Maximum Value in a Two-Dimensional Array

```
#include <iostream> // include input-output library using
```

```
namespace std; // use standard namespace int main() {
```

```
// main function starts
```

```
int matrix[4][4] = { {1,9,3,7}, {4,2,6,8}, {5,11,13,0}, {12,14,10,15} };
```

```
// declare and initialize 4x4 array int maxVal = matrix[0][0];
```

```
// assume first element is maximum for(int i = 0; i < 4; i++)
```

```
{ // loop for rows for(int j = 0; j < 4; j++) { // loop for
```

```
columns if(matrix[i][j] > maxVal) { // check if current
```

```
element is greater
```

```
maxVal = matrix[i][j]; // update maximum value
```

```
}
```

```
}
```

```
}
```

```
cout << "Maximum value in array = " << maxVal << endl; // display
maximum value

return 0;                // end program

}
```

4. Results

- Task 1 displayed the initialized 3x3 matrix with values from 1 to 9.
- Task 2 correctly calculated and displayed the sum of each row, each column, and the total sum of all elements.
- Task 3 successfully identified and displayed the maximum value in the 4x4 array.

These results confirm that two-dimensional arrays can store tabular data and allow operations like summation and searching for maximum values.

5. Conclusion

In this lab, we explored two-dimensional arrays in C++. We learned how to declare, initialize, and access elements using row and column indices. We practiced operations such as summing rows and columns and finding the maximum value in a matrix.

Two-dimensional arrays are powerful tools for representing structured data. They are widely used in mathematics (matrices), computer graphics (images), and engineering applications (grids and tables). By mastering 2D arrays, students gain the ability to handle more complex problems involving multidimensional data. This lab provided practical experience that strengthens programming skills and prepares students for advanced topics like dynamic memory allocation and algorithm design.

